

CSE2315 — Assignment 6

Vlad Paun
6152937

March 22, 2026

Exercise 1

Suppose we have the following language:

$$E_{\text{GNFA}} = \{\langle G \rangle \mid G \text{ is a GNFA such that } L(G) = \emptyset\}.$$

Show that E_{GNFA} is decidable. You are allowed to make use of the deciders implied by the text in the book (e.g., Section 4.1).

Answer: We show that E_{GNFA} is decidable by giving decider M as follows:

$M =$ "On input $\langle G \rangle$:

1. Convert GNFA G to an equivalent DFA D
2. Run the decider for E_{DFA} on $\langle D \rangle$
3. If the decider accepts, accept. Otherwise, reject.

Exercise 2

Consider the language

$$L = \{\langle M, w \rangle \mid M \text{ halts on } w \text{ and moves the head at most } |w|^2 \text{ steps}\}.$$

Is this language decidable? Explain.

Answer: L is decidable. A TM must move the head every step. So we can simulate M for at most steps. If it halts before reaching this limit we can accept. Otherwise, M is either looping or it moves the head more than $|w|^2$ steps, so we can reject.

Exercise 3

Old exam question, endterm 20–21: Show the claim is true or give a counterexample and explanation to show the claim is false, starting your answer with either **True** or **False**.

If there is a Turing reduction from L to E_{CFG} , then $\bar{L}$ is Turing-recognisable.

True. If there is a reduction from L to E_{CFG} , that means that L is no harder than E_{CFG} . So if the latter is decidable, which it is, then so is the former. So L is also decidable. Decidability is closed under complement, therefore $\bar{L}$ is also decidable. Any decidable language is also Turing-recognizable.

Exercise 4

Old exam question, endterm 22–23:

Consider the following language:

$$L = \{\langle M, q \rangle \mid M \text{ is a Turing Machine and } q \text{ is a useless state in } M\}.$$

A useless state in a Turing Machine is one that is never entered on any input string.

Prove L is undecidable, using a Turing reduction $E_{TM} \leq_T L$.

Proof. Proof by contradiction.

Assume for the sake of contradiction that L is decidable. Then, there exists TM S that decides L . We can use M to show that E_{TM} is also decidable by constructing decider R :

$R =$ "On input $\langle M \rangle$:

1. Scan through the input and find the accept state of M , q_{accept}
2. Run S on $\langle M, q_{accept} \rangle$
3. If S accepts, accept, otherwise reject

Thus, if this machine accepts, the accept state of the input is useless so its language is empty and vice versa. So R correctly decides E_{TM} , which is a contradiction with theorem 5.2 $\square$

Exercise 5

Old exam question, resit 17–18: Suppose we have the following language

$$L = \{\langle M, v, G \rangle \mid M \text{ outputs } G \text{ on the tape, which is a context free grammar that accepts } v, \text{ when run on input } v.\}$$

We intend to prove that this language is undecidable. To that end we use a mapping reduction from $HALT_{TM}$ to L . The reduction f is computed by the following TM F :

$F =$ "On input $\langle M, w \rangle$:

1. Construct the following TM M' :

$M' =$ "On input x :

1. Clear the input from the tape.
 2. Run M on w .
 3. See question a.
 4. See question a.
- "

2. Write $\langle M', a, G \rangle$ to the output tape, where G is a grammar with one rule: $S \rightarrow a$ and starting variable S ."

(a) What should be done in steps 3 and 4 of M' ?

3. Generate a grammar G with one rule: $S \rightarrow a$ and starting variable S .
4. Output grammar G on the tape.

- (b) **Provide a proof showing f satisfies the requirements of a mapping reduction.**

Proof. F computes f , so f is a computable function. We also need to show that it is reliable. To this end, we look at two cases:

- If $\langle M, w \rangle \in HALT_{TM}$, then M halts on w and M' also halts and correctly writes the grammar G on input a .
- If $\langle M, w \rangle \notin HALT_{TM}$, then M never halts on w , so M' will never write the correct output on the tape.

□

Exercise 6

Old exam question, resit 20–21:

- (a) **Let MIA_{TM} be the language consisting of 3 TM's and a word such that the word is recognized by at least 2 of the 3 TM's. Show that this is a recognizable language.**

Formally described as

$$MIA_{TM} = \{ \langle M_1, M_2, M_3, w \rangle \mid M_1, M_2, M_3 \text{ are TM's and } w \in (L(M_1) \cap L(M_2)) \cup (L(M_1) \cap L(M_3)) \cup (L(M_2) \cap L(M_3)) \}.$$

Answer: We show this by constructing a TM R that recognizes MIA_{TM} :

$R =$ "On input $\langle M_1, M_2, M_3, w \rangle$:

1. Simulate M_1, M_2, M_3 on w in parallel
2. As soon as 2 of these machines accept, accept.
3. If 2 or more reject, reject."

- (b) **Show that MIA_{TM} is undecidable by constructing a Turing reduction from A_{TM} .**

Proof. Assume for the sake of contradiction that MIA_{TM} is decidable. Therefore, there exists TM S that decides MIA_{TM} . We then construct TM N as follows:

$N =$ "On input $\langle M, w \rangle$:

1. Create TM M_{none} that instantly rejects any input
2. Create TM M_{all} that instantly accepts any input
3. Run S on $\langle M, M_{none}, M_{all}, w \rangle$
4. If it accepts, accept, otherwise reject."

We can see that from the description of MIA_{TM} , S accepts w iff M accepts w and we have assumed that it is a decider so it always halts. Thus, N is a decider for A_{TM} , which is a contradiction. Thus, our assumption that MIA_{TM} is decidable must be false, so, by contradiction we have shown that MIA_{TM} is undecidable. □

(c) Show that MIA_{TM} is not co-Turing recognizable.

We have shown that MIA_{TM} is Turing recognizable. If $\overline{MIA_{TM}}$ were also Turing recognizable, then MIA_{TM} would be decidable, which we have shown it is not. Therefore, $\overline{MIA_{TM}}$ must be Turing unrecognizable, so MIA_{TM} is not co-Turing recognizable.

Exercise 7

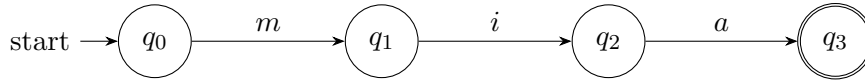
Revision, old exam question, midterm 19–20: Assume that we define an expanded NFA model by imposing the additional requirement that $|F| = 1$, and we call this an SNFA.

(a) Are the SNFA and NFA models equally expressive?

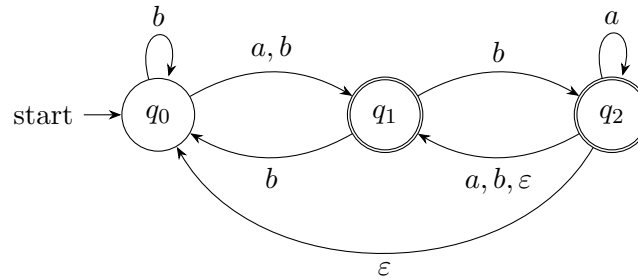
Start your answer with “Yes” or “No”. If you answer yes, explain how an arbitrary SNFA can be rewritten as an NFA and vice versa. If you answer no, give an example of a language that can be accepted by an NFA, but not by an SNFA or vice versa.

Yes. An *SNFA* is a special case of an *NFA*, so an arbitrary *SNFA* is already an *NFA*. To transform an arbitrary *NFA* to an *SNFA*, we add a new accepting state, we take all the old accepting states and make them not-accepting and add an epsilon-transition to the new single accepting state.

(b) Give an NFA of at most 4 states that accepts the word mia and rejects the word maya. No other restrictions on the language apply. A transition diagram suffices.



(c) Consider the NFA N depicted as:



Using the method from Sipser and leaving out unreachable states, construct a new DFA D , such that $L(D) = L(N)$. A transition diagram and a short explanation suffice.

